

IFN647 Week 11 Workshop: Text Classification & Clustering

Objectives:

- Understand how to **validate machine learning models** with tunable hyperparameters
 - Understand how **non-linear decision boundaries** are constructed via support vectors, and how `LinearSVC` **scores and ranks samples** using the `decision_function`
 - Understand how to **compute the probabilities** required by the Naïve Bayes (NB) classifier
 - Understand how to **design and implement an LDA-based clustering approach** for text data, and evaluate clustering performance using **purity** or **cluster precision**
-

Task 1. Understand how to Validate a Model with tunable parameters

The common validation strategy is cross validation which partitions (manually or randomly) a labelled data collection into **a training set** with $X\%$ of the observations and keeping the rest $(1-X\%)$ as **a test data set**. You can use this strategy to tune experimental parameters and update your model.

k-folds cross validation

This strategy divides the data set into k parts and uses each part one time as a test data set while using the others as a training data set. It has the advantage that we can use all the data available in the data collection.



E.g., $k = 5$

Experiment 1



Experiment 2



Experiment 3



Experiment 4



Experiment 5



Leave-1 out

This approach is the same as k -folds but with $k=n$, where n denotes the total number of examples. You always leave one observation out and train on the rest of the data. It is usually used only on small data sets, so it's more valuable to people evaluating laboratory experiments than to big data analysts.

Experiment 1



...

Experiment n



A systematic parameter search method (e.g., **grid search**) involves exhaustively evaluating all possible combinations of predefined parameter values within specified ranges. Each combination is assessed using a chosen evaluation metric on a validation set, and the configuration that yields the best performance is selected as the optimal parameter setting.

Systematic variation is a general approach: parameters are varied in a structured and controlled manner to observe their impact on performance. This may include changing one parameter at a time, heuristic exploration, or staged tuning.

(a) Assume you have the following setup:

- Dataset: 1000 documents + 10 queries with relevance judgments
- Evaluation metric: Mean Average Precision (MAP)
- Grid for λ :

$$\lambda \in \{0.0, 0.1, 0.2, \dots, 1.0\}$$

Please discuss how to use 5-fold cross-validation with grid search to tune the Jelinek–Mercer (JM) smoothing parameter in the following language model IR model.

$$p(q_i|D) = (1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|}$$

Where, λ is the smoothing parameter (to tune).

(b) Assume you want to tune the BM25 parameters k_1 and b for the parameter ranges

- $k_1 \in \{1.0, 1.2, 1.5, 2.0\}$
- $b \in \{0.5, 0.75, 1.0\}$

Discuss the steps or pseudocode used for optimal parameter setting based on (i) Grid search and (ii) Systematic variation.

Task 2. Linear and non-linear SVM classifiers

Given the following Multinomial Document Representation for 10 documents:

document <i>id</i>	cheap	buy	banking	dinner	the	<i>class</i>
1	0	0	0	0	2	not spam
2	3	0	1	0	1	spam
3	0	0	0	0	1	not spam
4	2	0	3	0	2	spam
5	5	2	0	0	1	spam
6	0	0	1	0	1	not spam
7	0	1	1	0	1	not spam
8	0	0	0	0	1	not spam
9	0	0	0	0	1	not spam
10	1	1	0	1	2	not spam

Let $D = \{d_1, d_2, \dots, d_{10}\}$ be a training set, and $C = \{\text{spam}, \text{not spam}\}$

Please use the Python **scikit-learn** package to train both non-linear and linear SVM classifiers using datasets D and C .

- Let X_{train} denote the list of document feature vectors derived from dataset D , and y_{train} denote the corresponding label vector, where “1” represents spam and “-1” represents non-spam. Provide the Python representations of X_{train} and y_{train} .
- Implement Python code to train: a non-linear SVM classifier, and a linear SVM classifier, using X_{train} and y_{train} . Then evaluate both models on the following test set: $X_{\text{test}} = [[1, 1, 1, 1, 2], [3, 0, 2, 0, 3], [0, 0, 0, 0, 0], [6, 0, 5, 0, 1], [4, 2, 0, 2, 1], [0, 0, 1, 1, 1], [0, 1, 0, 0, 1], [1, 0, 0, 0, 1], [0, 1, 0, 0, 1], [1, 1, 0, 1, 2]]$. Print the decision vectors of the non-linear SVM, and the decision function values of the linear SVM.
- Print the corresponding confusion matrix by using the corresponding justifications y_{test} , where $y_{\text{test}} = [-1, 1, 1, 1, 1, -1, -1, -1, -1, -1]$. Show the values of TP, FP, FN, and TN, and calculate the Accuracy.

Task 3. In week 9 review questions 2, we implemented a python function `TRAIN_MULTINOMIAL_NB(C, D, V)` to calculate prior probability $P(c)$ and conditional probabilities $P(w_i|c)$.

- Train a Multinomial NB classifier using function `TRAIN_MULTINOMIAL_NB()`, X_{train} and y_{train} of Task 2 to return (prior, condprob).
- Design a Python function `Test_MNB(prior, condprob, d)` that uses the returned (prior, condprob) to assign a label (1 or -1) to a document d .
- Test `Test_MNB(prior, condprob, d)` for all documents in X_{test} and put the results in variable ‘test_result’.
- Design list comprehensions to compute TP, TN, FP, FN by using ‘y_test’ and ‘test_result’, and then compute the accuracy of the multinomial NB classifier.

Task 4. Design and implement an LDA-based clustering approach to group reviews into three clusters and evaluate the results using purity. This task uses the Amazon Reviews labeled dataset, which contains 20,000 reviews. The dataset is structured as follows:

	reviewText	Positive
0	This is a one of the best apps according to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1
...	...	...
19995	this app is fricken stupid.it froze on the kin...	0
19996	Please add me!!!! I need neighbors! Ginger101...	1
19997	love it! this game. is awesome. wish it had m...	1
19998	I love love love this app on my side of fashio...	1
19999	This game is a rip off. Here is a list of thin...	0

20000 rows x 2 columns

- Load the dataset of Amazon reviews using `pd.read_csv()` and define a function `parse_text()` to preprocess the `reviewText` column.
- Convert the `reviewText` column into a list of Python strings, `docs`, and the `positive` column into a list of Python integers, `y_true`
- Design an LDA-based clustering model using `LatentDirichletAllocation` and `CountVectorizer`
- Apply the model to `docs`, and implement a purity score function to evaluate the clustering performance.